

HOPE LEARNING

Content Based Recommendation System: Technical Writeup

(by Dr. N. Dilip Raja)

This document provides a comprehensive explanation of the Content-Based Recommendation System code. Unlike User-Based and Item-Based collaborative filtering approaches, this system makes recommendations by analyzing the **content features** (movie genres) rather than relying on user rating patterns. The system creates user preference profiles based on the genres of movies they have rated and recommends new movies that match those genre preferences.

File Information

- **Source File:** MultipleFiles/3 Content Based Recommendation System.py
- **Purpose:** Content-Based Collaborative Filtering for Movie Recommendations
- **Dataset:** MovieLens 100k
- **Method:** Uses movie genre attributes (content features) to determine recommendations

Step-by-Step Processes

1. Import Required Libraries

The code begins by importing essential Python libraries:

- **pandas:** Used for data frame operations and handling structured data
- **numpy:** Used for numerical computations and array manipulations

These libraries provide the foundational tools for data manipulation and mathematical operations required throughout the recommendation process.

2. Load Movie Dataset

The system loads the MovieLens 100k item dataset containing movie information:

- **File:** ml-100k/u.item
- **Separator:** Pipe-delimited file (|)
- **Column Names:**
 - Movie ID
 - Movie Title
 - Release Date
 - Video Release Date
 - IMDb URL

- Genre columns: Unknown, Action, Adventure, Animation, Children's, Comedy, Crime, Documentary, Drama, Fantasy, Film-Noir, Horror, Musical, Mystery, Romance, Sci-Fi, Thriller, War, Western
- **Encoding:** Latin-1 encoding

The dataset is loaded into a pandas DataFrame and the first 4 rows are displayed for verification.

3. Select Essential Movie Content

The code extracts only the essential columns for recommendation:

- **Selected Columns:** Movie ID, Movie Title, and all 19 genre columns
- Creates **Movie_content** DataFrame containing:
 - Movie ID (identifier)
 - Movie Title (readable name)
 - Genre binary indicators (1 or 0 for each genre)

This creates a clean dataset with movie features ready for content-based analysis.

4. Load Ratings Dataset

The system loads the user ratings dataset:

- **File:** ml-100k/u.data
- **Separator:** Tab-delimited file (`\t`)
- **Column Names:** User ID, Item ID, Rating, Timestamp
- **Encoding:** Latin-1 encoding

This dataset contains all user-movie ratings including user IDs, movie IDs, numerical ratings, and timestamps.

5. Verify Ratings Data

The code performs a verification check to ensure data is loaded correctly:

python

Copy code

```
Ratings[Ratings['user id']==196][['item id','rating']]
```

This filters and displays all ratings from User ID 196 to verify the data was loaded properly from the user ID dataset.

6. Create Pivot Table (User-Movie Matrix)

A pivot table is created representing the user-movie rating matrix:

- **Index:** User ID
- **Columns:** Movie ID

- **Values:** Ratings
- Uses **Ratings.pivot_table()** function
- Creates a sparse matrix where:
 - Rows represent individual users (943 users)
 - Columns represent movies (1682 movies)
 - Cell values contain the ratings users gave to movies

This matrix serves as the foundation for creating user vectors.

7. Data Preprocessing - Handle Missing Values

The pivot table undergoes preprocessing to handle missing values:

- **NaN Replacement:** Missing values (NaN) are replaced with 0 using **Data_Matrix = Data_Matrix.replace(np.nan, 0)**
- This ensures mathematical operations can proceed without errors from missing data
- Users who haven't rated certain movies are represented as 0 (no rating)

8. Create User Vector

The code creates a user vector by transposing the data matrix:

python

Copy code

```
User_Vector = Data_Matrix.T
```

- Transposes the Data Matrix so movies become rows and users become columns
- This format allows the system to work with each movie's rating history
- Dimensions: Movies (rows) × Users (columns)

9. Create Movie Vector

The code creates a movie vector containing genre information:

python

Copy code

```
Movie_Vector = Items[['Action','Adventure','Animation','Children\'s','Comedy','Crime','Documentary','Drama','Fantasy','Film-Noir','Horror','Musical','Mystery','Romance','Sci-Fi','Thriller','War','Western']]
```

- Extracts only the 19 genre columns from the movie items
- Each row represents a movie
- Each column represents a genre (binary: 1 if movie belongs to genre, 0 otherwise)

10. Reindex Movie Vector

The movie vector index is reset:

```
Movie_Vector.index = range(1,1683)
```

- Assigns new sequential indices from 1 to 1682
- This aligns the movie IDs with array positions for easier indexing
- Ensures consistent reference between different data structures

11. Transpose Movie Vector

The movie vector is transposed:

```
Movie_Vector = Movie_Vector.T
```

- Now columns represent movies and rows represent genres
- Dimensions: Genres (rows) × Movies (columns)
- This format allows dot product operations with user vectors

12. Create Content for Individual User

The code demonstrates how to create a content vector for a single user:

```
U1 = Ratings[Ratings['user id']==1][['item id','rating']]
```

- Filters ratings for User ID 1
- Extracts only the item IDs and ratings they gave

This is the initial step in building a personalized user profile.

13. Extract Movie Features for User

The code retrieves genre features for each movie the user has watched:

```
U1_list = []
```

```
for i in U1['item id']:
```

```
    F = Movie_Vector[i].values
```

```
    U1_list.append(F)
```

- Iterates through each movie ID the user has rated
- Retrieves the genre vector for each movie from Movie_Vector
- Collects all genre vectors into a list

This creates a comprehensive feature set representing what the user has consumed.

14. Convert to DataFrame

The genre features are converted to a DataFrame:

```
U1a = pd.DataFrame(U1_list, columns = range(1,19))
```

- Creates a DataFrame where each row is a movie the user rated
- Columns represent the 18 genre categories
- Allows for aggregation operations across all rated movies

15. Create Single User Vector (Genre Aggregation)

The code aggregates genre preferences into a single vector:

```
Single_Vector = []
```

```
for j in range(1,19):
```

```
    g = sum(U1a[j]/272)
```

```
    Single_Vector.append(g)
```

- Iterates through each genre column (1-18)
- Calculates the average rating/proportion for each genre
- **272** appears to be the total number of movies rated by this user
- Creates a single preference vector representing the user's genre interests

This condensed vector represents the user's overall genre preference profile.

16. Create User Vector Function

A comprehensive function is defined to generate vectors for all users:

```
def User_Vector(Ratings,Movie_Vector):
```

```
    Vector = []
```

```
    for user in range(1,944):
```

```
        U1 = Ratings[Ratings['user id']==user][['item id','rating']]
```

```
        U1_list = []
```

```
        for i in U1['item id']:
```

```
            F = Movie_Vector[i].values
```

```
            U1_list.append(F)
```

```
        U1a = pd.DataFrame(U1_list, columns = range(1,19))
```

```
        Single_Vector = []
```

```
        for j in range(1,19):
```

```
            g = sum(U1a[j]/len(U1))
```

```
            Single_Vector.append(g)
```

```
        Vector.append(Single_Vector)
```

```
    return Vector
```

This function:

- Iterates through all 943 users
- For each user, retrieves all movies they rated
- Extracts genre features for each rated movie
- Aggregates into a single preference vector using average
- Returns a list of all user preference vectors

Key Difference: Uses `len(U1)` instead of fixed 272 to normalize based on each user's actual number of rated movies.

17. Generate All User Vectors

The function is executed to create vectors for all users:

```
UV = User_Vector(Ratings,Movie_Vector)
```

```
UV = pd.DataFrame(UV)
```

- Creates a DataFrame containing preference vectors for all 943 users
- Each row represents a user
- Each column represents a genre preference score
- Dimensions: Users (rows) × Genres (columns)

18. Verify Vector Dimensions

The code checks the shapes of both vectors:

- **User Vector Shape: (943, 18)** - 943 users with 18 genre preferences each
- **Movie Vector Shape: (18, 1682)** - 18 genres across 1682 movies

These dimensions verify compatibility for dot product operations.

19. Calculate Content Vector (Dot Product)

The code performs matrix multiplication to create the content vector:

```
Content_Vector = np.dot(UV,Movie_Vector)
```

- Computes the dot product between:
 - User preference vectors (943×18)
 - Movie genre vectors (18×1682)
- Result: Content vector matrix (943×1682)
- This represents predicted ratings based on genre matching
- Higher values indicate stronger genre match between user preference and movie content

20. Convert to Transposed DataFrame

The content vector is converted for user-friendly access:

```
Content_Vector = Content_Vector.T
```

- Transposes the matrix so movies become rows and users become columns
- Allows direct access to predictions for each user
- Creates a user-movie prediction matrix

21. Create Backup File

The code saves the content vector for future reference:

```
Content_Vector.to_csv("Content_Vector.csv", index = False)
```

- Exports the content vector to a CSV file
- Allows reloading without recalculating
- Useful for testing and validation

22. Individual User Check - Find Unwatched Movies

The code performs analysis for a specific user:

```
U2 = Ratings[Ratings['user id']==4]['item id'].values
```

- Retrieves all movie IDs watched by User ID 4
- Stored in **U2** for comparison

23. Filter High-Prediction Movies

The code identifies movies with high predicted ratings:

```
Val = Content_Vector[Content_Vector[4]>(Content_Vector[4].max()-0.5)][2]
```

- Filters movies where predicted rating is within 0.5 of the maximum
- **[4]** refers to column for User ID 4
- Creates a list of movies the user would likely enjoy

24. Exclude Already Watched Movies

The code filters out movies the user has already seen:

```
Recom = []
```

```
for i in Val.index:
```

```
    if i in U2:
```

```
        pass
```

```
    else:
```

```
        Recom.append(i)
```

- Iterates through high-prediction movies

- Checks if each movie is in the user's watch history (**U2**)
- Only adds unwatched movies to the recommendation list

25. Final Recommendations Output

The code generates the final recommendation list:

```
print(Recom)
```

- Displays the list of movie IDs recommended for User ID 4
- These are movies the user hasn't watched but would likely rate highly based on genre preferences

Summary

This Content-Based Recommendation System successfully:

1. Loads and preprocesses the MovieLens movie and rating datasets
2. Extracts content features (genres) for all movies
3. Creates a user-movie rating matrix
4. Builds individual user preference profiles based on genres of movies they've rated
5. Computes genre-based predicted ratings using dot product
6. Identifies movies with high predicted ratings for each user
7. Filters out already-watched movies to create final recommendations